Full-Stack Assignment-1

NAME : RADHIKA SINGLA UID : 23BAI70364

CLASS : 23AML-3A SUBMITTED TO : MR. AKASH MAHADEV

1. Benefits of Design Patterns in Frontend Development

Design patterns provide standardized solutions to common problems, acting as blueprints that can be customized to solve specific design issues in your code.
Reusability: Patterns encourage writing modular code that can be reused across different parts of an application, reducing redundancy.
Maintainability: By decoupling code logic (e.g., separating UI from business logic), patterns make the codebase easier to debug, update, and refactor without breaking other parts of the system.
Scalability: Standardized structures allow applications to grow in complexity without becoming unmanageable "spaghetti code."
Communication: They provide a shared vocabulary for developers. Saying "we should use a Singleton here" or "this is an Observer pattern" instantly conveys complex architectural ideas.
Best Practices: They encapsulate years of collective experience, helping developers avoid common pitfalls and anti-patterns.

2. Global State vs. Local State in React

State management is crucial for determining where data lives and how it is accessed.
Local State (UI State)
Definition: Data managed within a single component, typically using useState or useReducer. It is not needed by the rest of the app.
Scope: Confined to the component and its immediate children (via props).
Examples: Form input values, boolean toggles (e.g., isModalOpen), active tab index.
Persistence: Lost when the component unmounts.
Global State (App State)
Definition: Data shared across multiple, distinct parts of the component tree, often managed by Context API, Redux, or Zustand.
Scope: Accessible by any component in the application that subscribes to the store.
Examples: User authentication status, theme settings (Dark/Light mode), shopping cart data, fetched API data used on multiple pages.
Persistence: Persists as long as the app is running (or longer if synced with localStorage).

3. Comparison of Routing Strategies

Feature	Client-Side Routing (CSR)	Server-Side Routing (SSR)	Hybrid (Static/ISR)
Rendering	Browser downloads JS bundle and renders HTML dynamically.	Server generates full HTML for every request and sends it to the browser.	Pages are pre-built at build time (SSG) or updated incrementally (ISR).
Performance	Slower initial load (white screen), but fast subsequent navigation.	Fast initial load (First Contentful Paint), but slower page-to-page navigation.	Extremely fast initial load; relies on hydration for interactivity.
SEO	Harder for crawlers to index (requires JS execution).	Excellent; crawlers see full HTML content immediately.	Excellent; content is static HTML.
Server Load	Low; server mainly serves static assets and APIs.	High; server computes HTML for every user request.	Low to Medium; served via CDN, only revalidates when necessary.
Use Cases	Dashboards, SaaS apps, interactive tools behind login.	News sites, eCommerce (dynamic inventory), social media feeds.	Blogs, marketing sites, documentation, help centers.

4. Common Component Design Patterns

A. Container–Presentational Pattern

Concept: Separates logic (Container) from the UI (Presentational).

Container: Handles data fetching, state management, and passes data as props.

Presentational: Dumb components that purely render HTML based on props.

Use Case: A UserProfileContainer fetches user data and passes it to UserProfile which simply displays the avatar and name.

B. Higher-Order Components (HOC)

Concept: A function that takes a component and returns a new component with added functionality.

Structure: const EnhancedComponent = withAuth(WrappedComponent);

Use Case: Reusing logic like authentication checks (withAuth), logging (withLogger), or adding styles across unrelated components.

C. Render Props

Concept: A component with a prop (usually called render or children) that is a function returning a React element.

Structure: <MouseTracker render={({ x, y }) => Position: {x}, {y} } />

Use Case: Sharing stateful logic between components without lifting state up, such as tracking window size, scroll position, or mouse coordinates.

5. Responsive Navigation Bar (Material UI)

Below is the implementation using Material UI components:

```
import React, { useState } from 'react';
import {
  AppBar, Toolbar, Typography, Button, IconButton, Drawer,
  List, ListItem, ListItemText, Box, useTheme, useMediaQuery
} from '@mui/material';
import MenuIcon from '@mui/icons-material/Menu';

const Navbar = () => {
  const [mobileOpen, setMobileOpen] = useState(false);
  const theme = useTheme();
  // Breakpoint: 'md' usually corresponds to 900px
  const isMobile = useMediaQuery(theme.breakpoints.down('md'));

  const navItems = ['Dashboard', 'Projects', 'Team', 'Reports'];

  const handleDrawerToggle = () => {
    setMobileOpen(!mobileOpen);
  };

  const drawer = (
    <Box onClick={handleDrawerToggle} sx={{ textAlign: 'center' }}>
      <Typography variant="h6" sx={{ my: 2 }}>
        PM Tool
      </Typography>
      <List>
        {navItems.map((item) => (
          <ListItem button key={item}>
            <ListItemText primary={item} />
          </ListItem>
        ))}
      </List>
    </Box>
  );

  return (
    <Box sx={{ flexGrow: 1 }}>
      <AppBar position="static">
        <Toolbar>
          {/* Mobile Menu Icon */}
          {isMobile && (
            <IconButton
              color="inherit"
              aria-label="open drawer"
              edge="start"
```

```
            onClick={handleDrawerToggle}
            sx={{ mr: 2 }}
          >
            <MenuIcon />
          </IconButton>
        )}

        {/* Logo */}
        <Typography variant="h6" component="div" sx={{ flexGrow: 1 }}>
          PM Tool
        </Typography>

        {/* Desktop Links */}
        {!isMobile && (
          <Box>
            {navItems.map((item) => (
              <Button key={item} color="inherit">
                {item}
              </Button>
            ))}
          </Box>
        )}
      </Toolbar>
    </AppBar>

    {/* Mobile Drawer implementation */}
    <Box component="nav">
      <Drawer
        variant="temporary"
        open={mobileOpen}
        onClose={handleDrawerToggle}
        ModalProps={{ keepMounted: true }} // Better open performance on mobile
        sx={{
          display: { xs: 'block', md: 'none' },
          '& .MuiDrawer-paper': { boxSizing: 'border-box', width: 240 },
        }}
      >
        {drawer}
      </Drawer>
    </Box>
  </Box>
 );
};


export default Navbar;
```

6. Frontend Architecture for PM Tool

A. SPA Structure

We use a feature-based folder structure.

```
src/
├── app/              # Redux store setup
├── components/       # Shared UI components (Buttons, Inputs)
├── features/         # Feature-based modules
│   ├── auth/         # Login, Register, ProtectedRoute
│   ├── projects/     # Project lists, Board view
│   └── tasks/        # Task details, comments
├── hooks/            # Custom hooks (useDebounce, useWebSocket)
├── layout/           # MainLayout (Sidebar + Navbar)
├── theme/            # Material UI theme configuration
└── utils/            # Helpers, Constants
```

Routing (React Router v6):

```
// src/App.tsx
import { Routes, Route } from 'react-router-dom';
import ProtectedRoute from './features/auth/ProtectedRoute';
import Dashboard from './features/projects/Dashboard';
import ProjectDetails from './features/projects/ProjectDetails';

const App = () => {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route element={<ProtectedRoute />}>
        <Route path="/" element={<Dashboard />} />
        <Route path="projects">
          <Route path=":id" element={<ProjectDetails />} />
        </Route>
      </Route>
    </Routes>
  );
};
```

B. Global State Management (Redux Toolkit)

We use createSlice for synchronous actions and createAsyncThunk for API calls.

```
// src/features/projects/projectSlice.ts
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Thunk for fetching data
export const fetchProjects = createAsyncThunk(
  'projects/fetch',
  async (userId, thunkAPI) => {
    const response = await fetch(`/api/projects?user=${userId}`);
    return await response.json();
```

```
  }
);

const projectSlice = createSlice({
  name: 'projects',
  initialState: { items: [], status: 'idle' },
  reducers: {
    // Action for Real-time update via WebSocket
    updateProjectRealtime: (state, action) => {
      const index = state.items.findIndex(p => p.id === action.payload.id);
      if (index !== -1) {
        state.items[index] = action.payload;
      }
    }
  },
  extraReducers: (builder) => {
    builder.addCase(fetchProjects.fulfilled, (state, action) => {
      state.items = action.payload;
    });
  }
});

export const { updateProjectRealtime } = projectSlice.actions;
export default projectSlice.reducer;
```

C. Responsive UI Design (Material UI Theme)

Customizing the theme ensures consistency.

```
// src/theme/index.ts
import { createTheme } from '@mui/material/styles';

const theme = createTheme({
  palette: {
    primary: { main: '#1976d2' },
    background: { default: '#f4f6f8' },
  },
  components: {
    MuiButton: {
      styleOverrides: {
        root: { textTransform: 'none', borderRadius: 8 },
      },
    },
  },
});

export default theme;
```

D. Performance Optimization Techniques

1. **Code Splitting:** Use React.lazy and Suspense to load heavy routes (like the Kanban board) only when needed.

2. **Virtualization:** Use react-window or react-virtuoso for rendering large lists of tasks. Instead of rendering 1000 DOM nodes, it only renders the 20 visible ones.
3. **Memoization:** Use React.memo for list items (e.g., a single Task Card) to prevent re-renders when the parent (Project Board) updates but the task data hasn't changed.

E. Scalability & Real-Time Improvements

1. **WebSockets:** Instead of polling, use a WebSocket connection (e.g., Socket.io) to push updates.
    - *Implementation:* Create a custom middleware in Redux or a useWebSocket hook that dispatches updateProjectRealtime when a message is received.
2. **Optimistic UI:** When a user moves a task card, update the Redux store immediately before the API confirms it. If the API fails, roll back the change. This makes the app feel instant.
3. **Conflict Resolution:** For multi-user concurrent access (User A and User B edit the same task), implement "Last Write Wins" or Operational Transformation (OT) logic on the backend, and notify the frontend to re-fetch or merge data.